

PROJECT REPORT

EXPERIMENT - 8

Project Statement

Design and develop an MQTT-Based Remote HVAC Monitoring System using Thing Speak Cloud. The system remotely monitors HVAC parameters and stores sensor data on cloud platforms for analysis and control.

Submitted by

Abinaya S – 24MER001

Dhiyanasree KK – 24MER008

Puthiyaraj M – 24MER045

Rithick S – 24MER051

MQTT-Based Remote HVAC Monitoring System Using Thing Speak Cloud

Project Overview

The MQTT-Based Remote HVAC Monitoring System Using Thing Speak Cloud is an IoT-based system designed to remotely monitor temperature and humidity conditions in an HVAC environment. The system uses an ESP8266 Node MCU as the main controller and a DHT22 sensor to measure temperature and humidity.

The collected sensor data is processed by the Node MCU and transmitted through Wi-Fi using the MQTT communication protocol. The data can be integrated with the Thing Speak Cloud for remote monitoring, data storage, visualization, and analysis.

An LCD display provides local information about the measured environmental conditions, while a relay module can be used to control an HVAC-related load based on predefined temperature or humidity conditions.

The proposed system reduces the need for continuous manual monitoring and provides a low-cost solution for IoT-based HVAC monitoring and control.

Main Objectives:

The main objectives of the project are:

- To develop an IoT-based HVAC monitoring system.
- To measure temperature and humidity continuously using the DHT22 sensor.
- To use ESP8266 Node MCU for data processing and Wi-Fi communication.
- To transmit HVAC sensor data using the MQTT protocol.
- To store and visualize sensor data through Thing Speak Cloud.
- To provide real-time local readings using an LCD display.
- To control an HVAC-related load using a relay module.

- To enable remote monitoring of HVAC environmental conditions.
- To analyze temperature and humidity trends using cloud-stored data.
- To develop a low-cost and scalable HVAC monitoring prototype.

Problem Statement

Traditional HVAC systems often depend on local monitoring and manual inspection of temperature and humidity conditions. This makes it difficult to continuously monitor HVAC environments from a remote location.

The proposed system solves this problem by developing an MQTT-based remote HVAC monitoring system using Thing Speak Cloud. The system measures temperature and humidity using a DHT22 sensor, processes the readings through ESP8266 Node MCU, and sends the data through Wi-Fi using MQTT.

The sensor information is stored and visualized on the cloud, allowing users to remotely observe HVAC conditions and analyze environmental trends. A relay module can also be operated based on programmed threshold conditions to assist in HVAC load control.

Problems Addressed:

The proposed system addresses the following problems:

- Lack of continuous HVAC environmental monitoring.
- Difficulty in accessing HVAC parameters remotely.
- Manual recording of temperature and humidity data.
- Lack of historical environmental data for analysis.
- Delayed identification of abnormal temperature conditions.
- Limited visibility of HVAC conditions from a remote location.
- Lack of automatic load-control capability in basic monitoring systems.
- Increased effort required for manual inspection.

Proposed Solution

The proposed solution consists of an ESP8266 Node MCU, DHT22 temperature and humidity sensor, relay module, and LCD display.

The DHT22 continuously measures the temperature and humidity of the HVAC environment. The ESP8266 reads the sensor data and connects to a Wi-Fi network. The measured parameters are transmitted using MQTT-based communication and made available for cloud monitoring through Thing Speak.

The LCD display shows the current temperature and humidity locally. The relay module can be programmed to switch an HVAC-related load ON or OFF when the measured temperature crosses a predefined threshold.

Main functions:

- Temperature measurement
- Humidity measurement
- Wi-Fi connectivity
- MQTT communication
- Cloud data monitoring
- Local display
- Threshold-based relay control
- Historical data analysis

Proposed Operation:

The proposed system operates in the following sequence:

Step 1: Power is supplied to the ESP8266 Node MCU and other modules.

Step 2: The ESP8266 initializes the DHT22 sensor, LCD display, relay module, and Wi-Fi connection.

Step 3: The DHT22 measures the surrounding temperature and humidity.

Step 4: ESP8266 receives and processes the sensor readings.

Step 5: The temperature and humidity values are displayed on the LCD.

Step 6: ESP8266 establishes communication with the MQTT-based cloud communication system through Wi-Fi.

Step 7: Sensor readings are transmitted for cloud monitoring and storage through Thing Speak.

Step 8: The temperature value is compared with a predefined threshold.

Step 9: If the temperature exceeds the selected threshold, the relay can be activated to control the connected HVAC load.

Step 10: The process continuously repeats at regular intervals, allowing real-time monitoring.

System Architecture

The system can be divided into five major sections:

Input Section:

The DHT22 sensor acts as the input device.

It measures:

- Temperature
- Relative humidity

Processing Section:

The ESP8266 Node MCU acts as the central controller.

It performs:

- Sensor data acquisition
- Data processing
- Threshold comparison
- Wi-Fi communication
- MQTT communication
- Control decisions

Communication Section:

The built-in Wi-Fi capability of the ESP8266 provides wireless connectivity.

MQTT is used as the lightweight communication protocol for transmitting sensor information.

Output and Actuation Section:

The system contains:

- LCD display for local monitoring
- Relay module for controlling a connected HVAC-related load

Overall Flow:

DHT22 Sensor → ESP8266 Node MCU → Wi-Fi → MQTT → Thing Speak Cloud

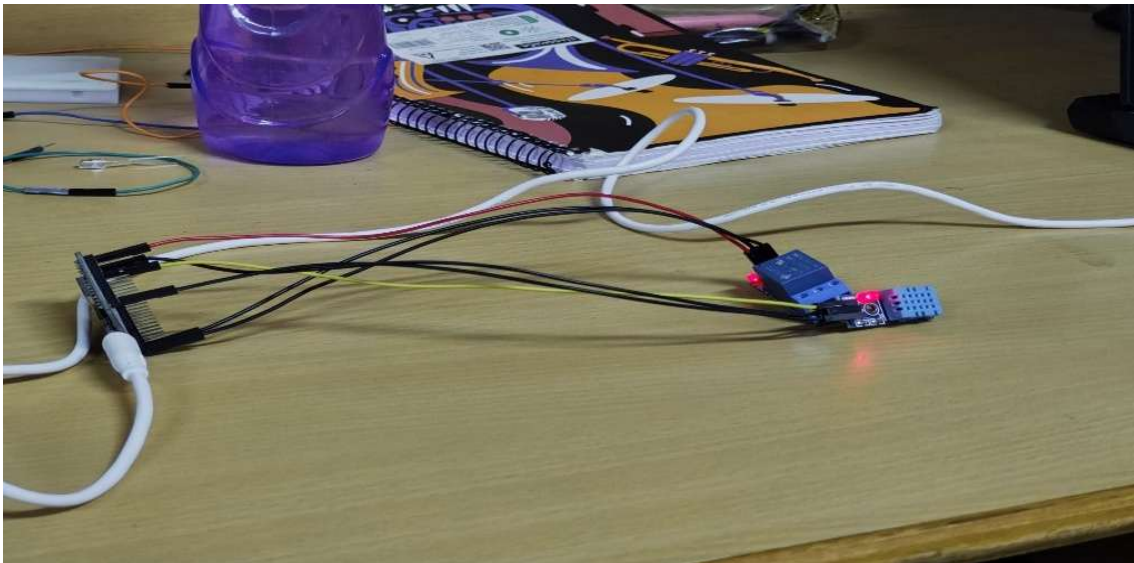
At the same time:

ESP8266 → LCD Display

and

ESP8266 → Relay Module → HVAC Load

Image:



Circuit Table

S. No	Component	Pin / Terminal	ESP8266 Node MCU Connection	Function
1	DHT22 Sensor	VCC	3.3V	Provides power to the sensor
		DATA	D5 (GPIO14)	Sends temperature and humidity data
		GND	GND	Common ground
2	Relay Module	VCC	VIN/5V supply*	Provides power to relay
		IN	D1 (GPIO5)	Controls relay ON/OFF
		GND	GND	Common ground
3	ESP8266 Node MCU	USB/VIN	USB/appropriate power supply	Powers the controller
4	Breadboard	Power Rails	3.3V/VIN and GND	Provides common power connections
5	Jumper Wires	—	Between modules	Provides circuit interconnections

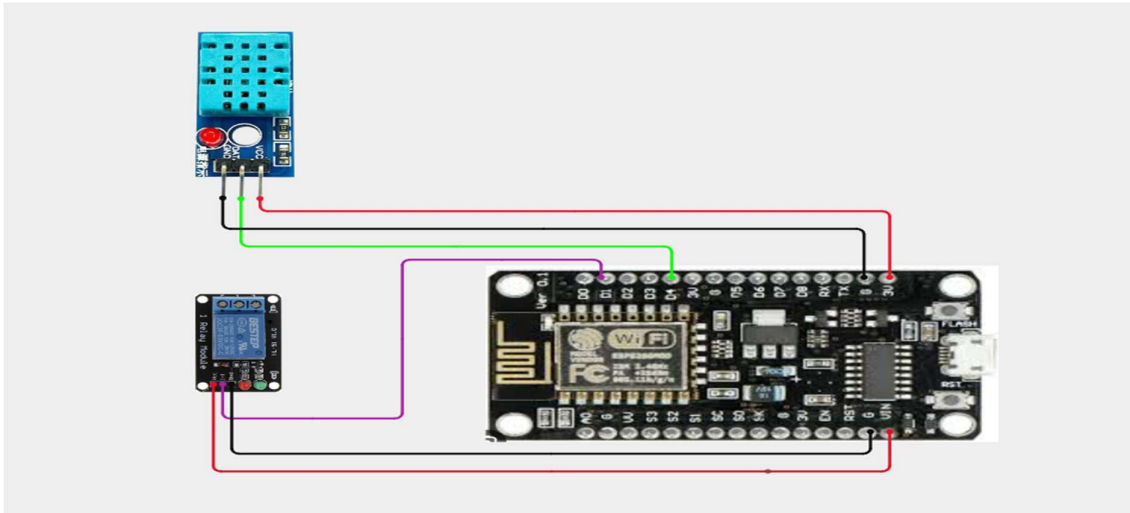
Important Pin Mapping

A suitable example pin mapping for an ESP8266 Node MCU is:

- DHT22 DATA → D5
- Relay IN → D1
- All GND terminals → Common GND

- DHT22 VCC → 3.3V

Circuit design



Algorithm

Step 1:Start the system.

Step 2:Initialize ESP8266 Node MCU.

Step 3:Initialize DHT22 sensor.

Step 4:Initialize LCD display.

Step 5:Initialize relay output.

Step 6:Connect ESP8266 to the Wi-Fi network.

Step 7:Establish MQTT communication.

Step 8:Read temperature and humidity from DHT22.

Step 9:Check whether the sensor readings are valid.

Step 10:Display temperature and humidity on the LCD.

Step 11:Send the sensor data through MQTT for cloud monitoring.

Step 12:Update the corresponding Thing Speak data fields.

Step 13: Compare the temperature with the predefined threshold.

Step 14: Activate or deactivate the relay according to the programmed control condition.

Step 15: Wait for the next measurement interval.

Step 16: Repeat the process continuously.

Coding

```
#include <ESP8266WiFi.h>
```

```
#include <PubSubClient.h>
```

```
#include <DHT.h>
```

```
// Wi-Fi Credentials
```

```
const char* ssid = "Rithick";
```

```
const char* password = "12345678";
```

```
// ThingSpeak MQTT Configuration
```

```
const char* mqttServer = "mqtt3.thingspeak.com";
```

```
const int mqttPort = 1883;
```

```
const char* clientID = "Hg8CDwMKOwM3HqYUNRA1Exg";
```

```
const char* mqttUsername = "Hg8CDwMKOwM3HqYUNRA1Exg";
```

```
const char* mqttPassword = "1rKxvN+O0/BEpg22MF7ja+4D";
```

```
const char* channelID = "3459607";
```

```
// Hardware Configuration (Corrected NodeMCU Pin Map)
```

```
#define DHTPIN 2          // GPIO 2 is silk-screen labeled D4 on NodeMCU
```

```
#define DHTTYPE DHT11
```

```
#define RELAY_PIN 5      // GPIO 5 is silk-screen labeled D1 on NodeMCU
```

```
DHT dht(DHTPIN, DHTTYPE);
```

```
WiFiClient espClient;
```

```
PubSubClient client(espClient);
```

```
unsigned long lastMsg = 0;
```

```
int relayStatus = 1;      // Default: HVAC ON
```

```
void setupWiFi() {
```

```
    delay(10);
```

```
    Serial.println();
```

```
    Serial.print("Connecting to Wi-Fi Network: ");
```

```
    Serial.println(ssid);
```

```
    WiFi.mode(WIFI_STA);
```

```
    WiFi.begin(ssid, password);
```

```
    while (WiFi.status() != WL_CONNECTED) {
```

```
        delay(500);
```

```
        Serial.print(".");
```

```
    }
```

```
    Serial.println("\n[OK] Wi-Fi Connected successfully!");
```

```
    Serial.print("NodeMCU Assigned IP: ");
```

```
    Serial.println(WiFi.localIP());
```

```
}
```

```
void reconnect() {  
  while (!client.connected()) {  
    Serial.print("Connecting to ThingSpeak MQTT Broker...");  
  
    if (client.connect(clientID, mqttUsername, mqttPassword)) {  
      Serial.println("\n[OK] Connected to ThingSpeak MQTT Server!");  
    } else {  
      Serial.print("\n[ERROR] Connection failed. Reason code (rc) = ");  
      Serial.print(client.state());  
      Serial.println(". Retrying in 5 seconds...");  
      delay(5000);  
    }  
  }  
}
```

```
void setup() {  
  Serial.begin(115200);  
  
  pinMode(RELAY_PIN, OUTPUT);  
  digitalWrite(RELAY_PIN, LOW); // LOW activates most relay boards  
  
  dht.begin();  
  setupWiFi();  
  client.setServer(mqttServer, mqttPort);
```

```
}
```

```
void loop() {
```

```
  if (!client.connected()) {
```

```
    reconnect();
```

```
  }
```

```
  client.loop();
```

```
  // Publish telemetry every 20 seconds
```

```
  unsigned long now = millis();
```

```
  if (now - lastMsg > 20000) {
```

```
    lastMsg = now;
```

```
    float temp = dht.readTemperature();
```

```
    float hum = dht.readHumidity();
```

```
    if (isnan(temp) || isnan(hum)) {
```

```
      Serial.println("[WARNING] Failed to read data from DHT11 sensor! Check  
pin connections.");
```

```
      return;
```

```
    }
```

```
    // High temperature emergency shutdown check (> 35°C threshold)
```

```
    if (temp > 30.0) {
```

```
      digitalWrite(RELAY_PIN, HIGH); // Turn OFF HVAC relay
```

```

    relayStatus = 0;
} else {

    digitalWrite(RELAY_PIN, LOW); // Keep HVAC ON
    relayStatus = 1;
}

// Output reading to Serial Monitor
Serial.print("Temperature: ");
Serial.print(temp, 1);
Serial.print(" °C | Humidity: ");
Serial.print(hum, 1);
Serial.print(" % | Relay State: ");
Serial.println(relayStatus);

// Prepare MQTT Topic & Payload
String topic = "channels/" + String(channelID) + "/publish";
String payload = "field1=" + String(temp, 1) +
    "&field2=" + String(hum, 1) +
    "&field3=" + String(relayStatus);

Serial.print("Publishing to topic [");
Serial.print(topic);
Serial.print("]: ");
Serial.println(payload);

```

```

if (client.publish(topic.c_str(), payload.c_str())) {

    Serial.println("[OK] Packet published to ThingSpeak!");

} else {

    Serial.println("[ERROR] Failed to publish packet.");

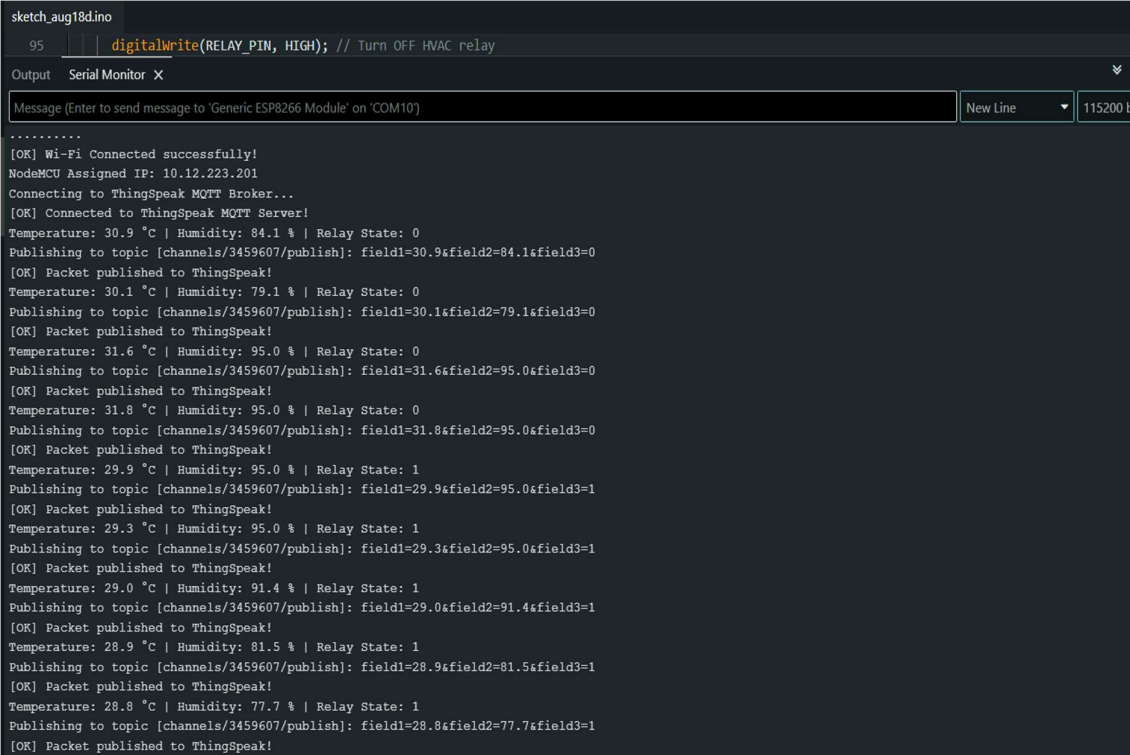
}

}

}

```

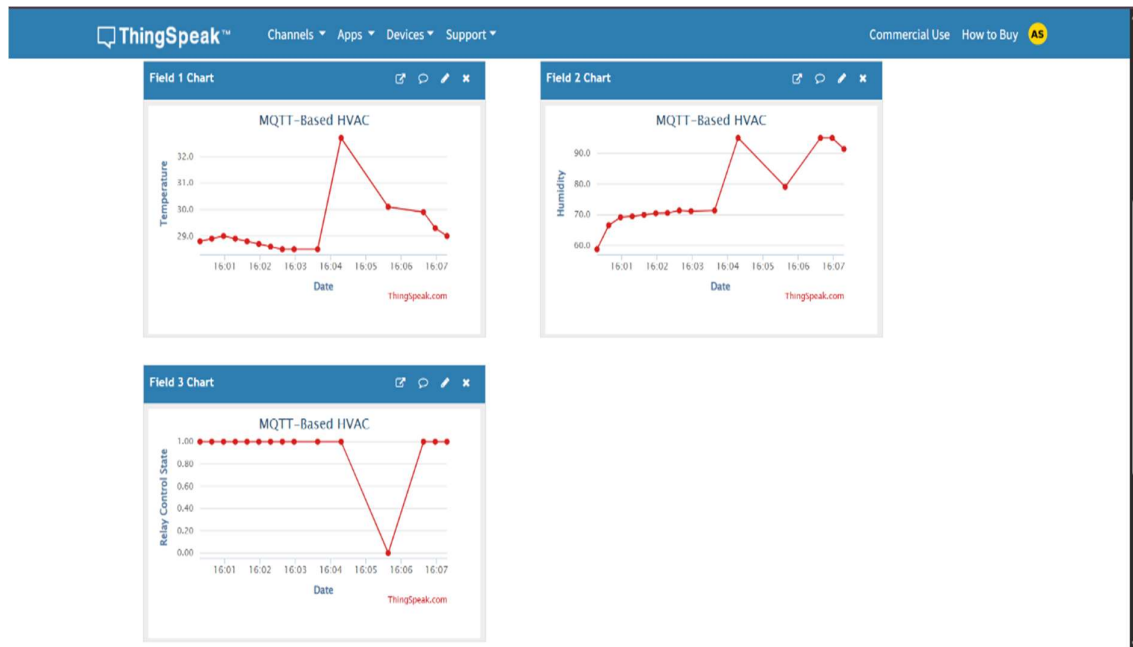
Coding Output:



```

sketch_aug18d.ino
95 | | | digitalWrite(RELAY_PIN, HIGH); // Turn OFF HVAC relay
Output Serial Monitor X
Message (Enter to send message to 'Generic ESP8266 Module' on 'COM10')
-----
[OK] Wi-Fi Connected successfully!
NodeMCU Assigned IP: 10.12.223.201
Connecting to ThingSpeak MQTT Broker...
[OK] Connected to ThingSpeak MQTT Server!
Temperature: 30.9 °C | Humidity: 84.1 % | Relay State: 0
Publishing to topic [channels/3459607/publish]: field1=30.9&field2=84.1&field3=0
[OK] Packet published to ThingSpeak!
Temperature: 30.1 °C | Humidity: 79.1 % | Relay State: 0
Publishing to topic [channels/3459607/publish]: field1=30.1&field2=79.1&field3=0
[OK] Packet published to ThingSpeak!
Temperature: 31.6 °C | Humidity: 95.0 % | Relay State: 0
Publishing to topic [channels/3459607/publish]: field1=31.6&field2=95.0&field3=0
[OK] Packet published to ThingSpeak!
Temperature: 31.8 °C | Humidity: 95.0 % | Relay State: 0
Publishing to topic [channels/3459607/publish]: field1=31.8&field2=95.0&field3=0
[OK] Packet published to ThingSpeak!
Temperature: 29.9 °C | Humidity: 95.0 % | Relay State: 1
Publishing to topic [channels/3459607/publish]: field1=29.9&field2=95.0&field3=1
[OK] Packet published to ThingSpeak!
Temperature: 29.3 °C | Humidity: 95.0 % | Relay State: 1
Publishing to topic [channels/3459607/publish]: field1=29.3&field2=95.0&field3=1
[OK] Packet published to ThingSpeak!
Temperature: 29.0 °C | Humidity: 91.4 % | Relay State: 1
Publishing to topic [channels/3459607/publish]: field1=29.0&field2=91.4&field3=1
[OK] Packet published to ThingSpeak!
Temperature: 28.9 °C | Humidity: 81.5 % | Relay State: 1
Publishing to topic [channels/3459607/publish]: field1=28.9&field2=81.5&field3=1
[OK] Packet published to ThingSpeak!
Temperature: 28.8 °C | Humidity: 77.7 % | Relay State: 1
Publishing to topic [channels/3459607/publish]: field1=28.8&field2=77.7&field3=1
[OK] Packet published to ThingSpeak!

```



System Working

The system starts when power is supplied to the ESP8266 Node MCU. The controller initializes all connected components and establishes a Wi-Fi connection.

The DHT22 sensor continuously measures the temperature and humidity of the surrounding HVAC environment. These measurements are transferred to the ESP8266 through the sensor's data line.

The ESP8266 processes the received values and displays them on the LCD screen. The controller then communicates through Wi-Fi and uses MQTT-based communication to transmit the readings for remote cloud monitoring.

Thing Speak is used to store and visualize the sensor data. Users can observe temperature and humidity values remotely and analyze their variation over time.

The ESP8266 also compares the measured temperature with a predefined threshold. When the temperature reaches the specified condition, the relay output is changed accordingly. The relay can be connected to an appropriate HVAC control interface or low-voltage demonstration load.

The monitoring and control cycle continues automatically at regular intervals.

Bill of Materials

S. No	Component	Specification	Quantity
1	ESP8266 Node MCU	Wi-Fi-enabled microcontroller	1
2	DHT22 Sensor	Digital temperature & humidity sensor	1
3	Relay Module	1-channel relay module	1
4	Breadboard	Standard solderless breadboard	1
5	Jumper Wires	Male-to-male / male-to-female	1 set
6	USB Cable	Micro-USB/compatible with Node MCU	1
7	Power Supply	5 V USB power source	1
8	Computer/Laptop	Arduino IDE + Internet access	1
9	Wi-Fi Network	2.4 GHz Wi-Fi	1

Prototype Implementation

The prototype is implemented by connecting the DHT22 sensor, relay module, and LCD display to the ESP8266 Node MCU on a breadboard.

The DHT22 is positioned so that it can sense the environmental conditions around the HVAC demonstration area. The ESP8266 is connected to a computer using a USB cable for programming and power.

The LCD is connected using the appropriate communication interface, while the relay module is connected to a digital output of the ESP8266.

After programming, the Node MCU connects to the available Wi-Fi network. The measured temperature and humidity are then transmitted through the MQTT communication mechanism and monitored through the Thing Speak cloud environment.

Hardware Setup:

The hardware setup consists of the following arrangement:

DHT22 Sensor

- Connected to ESP8266 digital input
- Measures temperature and humidity

ESP8266 Node MCU

- Central processing unit
- Wi-Fi connectivity
- MQTT communication
- Relay control

Relay Module

- Connected to ESP8266 digital output
- Controls the demonstration HVAC/load interface

Breadboard

- Provides convenient prototyping connections

Jumper Wires

- Used for electrical connections between modules

Prototype Working

When the prototype is powered ON, the ESP8266 initializes the connected sensors and display.

The DHT22 measures the current temperature and humidity. The values are processed by the ESP8266 and displayed locally.

The ESP8266 then sends the sensor readings through the wireless network using MQTT communication. The data is made available on Thing Speak for remote visualization and storage.

If the measured temperature crosses the programmed threshold, the relay output changes state. This can represent the switching operation of an HVAC cooling or ventilation system in the prototype.

The system continuously repeats these operations, creating a simple remote HVAC monitoring and control platform.

Expected Prototype Output

The expected prototype output includes:

- Successful ESP8266 Wi-Fi connection.
- Continuous temperature measurement.
- Continuous humidity measurement.
- Temperature and humidity values displayed on LCD.
- Successful MQTT communication.
- Sensor data transmitted to the cloud.
- Temperature and humidity values visible on Thing Speak.
- Historical sensor data available for analysis.
- Relay switching based on the programmed threshold.
- Continuous remote HVAC environmental monitoring.

Prototype Demonstration

The prototype demonstration can be performed as follows:

Demonstration 1 – Temperature Monitoring

- Expose the DHT22 to different environmental conditions and observe the change in temperature on the LCD.

Demonstration 2 – Humidity Monitoring

- Observe the humidity value measured by the DHT22 and verify that it changes according to the surrounding environment.

Demonstration 3 – Cloud Monitoring

- Connect the ESP8266 to Wi-Fi and demonstrate the transmission of sensor values to Thing Speak.

Demonstration 4 – Data Visualization

- Show the temperature and humidity graphs on Thing Speak and demonstrate the stored historical readings.

Demonstration 5 – Relay Control

- Change the environmental temperature or use a predefined test condition and demonstrate the relay switching operation.

Results & Performance Analysis

The MQTT-Based Remote HVAC Monitoring System Using Thing Speak Cloud was successfully designed as an IoT-based monitoring prototype. The ESP8266 Node MCU collects temperature and humidity data from the DHT22 sensor and transmits the readings through Wi-Fi using MQTT communication.

The measured parameters are displayed locally on the LCD and can be monitored remotely through Thing Speak Cloud. The relay module provides basic control based on a predefined temperature threshold.

Parameter	Observed/Expected Result
Temperature Monitoring	Continuous measurement using DHT22
Humidity Monitoring	Continuous measurement using DHT22
Local Display	Temperature and humidity displayed on OLED/LCD
Wi-Fi Connectivity	Wireless communication established
MQTT Communication	Sensor data transmitted through MQTT
Cloud Monitoring	Data available on Thing Speak
Data Visualization	Temperature and humidity trends displayed as graphs
Relay Control	ON/OFF operation based on programmed threshold
Remote Monitoring	HVAC conditions accessible through cloud
Data Storage	Sensor readings stored for further analysis

Expected Results:

The expected results of the project are:

- Accurate measurement of HVAC temperature and humidity.
- Successful wireless communication through ESP8266.
- Successful MQTT-based data transmission.
- Remote visualization of HVAC parameters.
- Cloud-based storage of sensor readings.
- Real-time local display of environmental conditions.
- Automatic relay operation according to the selected threshold.
- Reduced requirement for manual monitoring.
- Availability of historical data for HVAC analysis.
- Demonstration of an IoT-enabled HVAC monitoring system.

Performance Analysis

Temperature Monitoring:

The DHT22 provides digital temperature readings that can be periodically collected by the ESP8266. The readings can be compared with reference measurements to evaluate the accuracy of the prototype.

Humidity Monitoring:

Humidity readings are continuously obtained from the DHT22 and transmitted to the cloud. Changes in humidity can be observed using the ThingSpeak visualization.

Communication Performance:

ESP8266 provides Wi-Fi connectivity for remote data transmission. MQTT is suitable for IoT applications because it uses a lightweight publish/subscribe communication model.

Cloud Performance:

Thing Speak provides a convenient platform for collecting and visualizing the sensor readings. Temperature and humidity trends can be represented using graphical plots.

Relay Response:

The relay operation can be evaluated by applying different temperature conditions and observing whether the relay changes state according to the programmed threshold.